Laptop Price Prediction

Tsung-Yu Lee

2024-06-21

Table of contents

Introduction

Data Source

The laptop dataset is from Kaggle Laptop Sales Price Prediction.

```r
setwd("C:/github_LTY/homework/Final Report") # Set working directory.
laptop <- read.csv("data/laptop.csv", na.strings = c("","NA"))
```

Because "X" and "Name" are useless, so we delete these columns in advance.

```r
laptop <- subset(x = laptop, select = -c(X, Name))
```

Summary Statistics

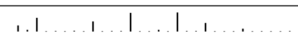
```r
library(Hmisc) # summary statistics
```

```
Attaching package: 'Hmisc'


The following objects are masked from 'package:base':

    format.pval, units
```

```r
latex(describe(laptop), file = "", caption.placement = "top")
```

laptop
27 Variables 1020 Observations

Brand



n	missing	distinct
1020	0	31

```
lowest : Acer      Apple     Asus      ASUS      Avita
highest: Ultimus   Walker    Wings     Xiaomi    Zebronics
```

Price

n	missing	distinct	Info	Mean	Gmd	.05	.10	.25	.50	.75	.90	.95
1020	0	561	1	82063	59619	25598	31990	43990	63690	94990	149990	199990

```
lowest :   8000   12990   13990   14490   14990, highest: 450990 453990 489883 519990 599990
```

Rating

n	missing	distinct	Info	Mean	Gmd	.05	.10	.25	.50	.75	.90	.95
1020	0	17	0.996	4.374	0.2689	4.00	4.05	4.20	4.35	4.55	4.70	4.75

```
Value       3.95  4.00  4.05  4.10  4.15  4.20  4.25  4.30  4.35  4.40  4.45  4.50  4.55  4.60
Frequency      1    71    56    69    57    66    68    57    71    76    55    57    64    57
Proportion 0.001 0.070 0.055 0.068 0.056 0.065 0.067 0.056 0.070 0.075 0.054 0.056 0.063 0.056

Value       4.65  4.70  4.75
Frequency     53    64    78
Proportion 0.052 0.063 0.076

For the frequency table, variable is rounded to the nearest 0
```

Processor_brand

n	missing	distinct
1020	0	7

Value	AMD	Apple	HiSilicon	Intel	MediaTek	Microsoft	Qualcomm
Frequency	250	18	1	742	7	1	1
Proportion	0.245	0.018	0.001	0.727	0.007	0.001	0.001

Processor_name

n	missing	distinct
1020	0	36

```
lowest : AMD Athlon        AMD Athlon Pro    AMD Athlon Silver AMD Ryzen 3       AMD Ryzen 5
highest: Mediatek          MediaTek          MediaTek Kompanio Microsoft SQ1 SQ1 Qualcomm X Elite
```

Processor_variant

n	missing	distinct
996	24	125

```
lowest : 1005G1    100U      10210U    10210Y    1035G1
highest: N5100     N6000     processor SQ1       Z3735F
```

Processor_gen

n	missing	distinct	Info	Mean	Gmd	.05	.10	.25	.50	.75	.90	.95
891	129	13	0.95	10.45	3.086	5	5	7	12	13	13	13

Value	1	3	4	5	6	7	8	9	10	11	12	13	14
Frequency	1	11	4	92	17	120	7	1	13	113	246	255	11
Proportion	0.001	0.012	0.004	0.103	0.019	0.135	0.008	0.001	0.015	0.127	0.276	0.286	0.012

For the frequency table, variable is rounded to the nearest 0

Core_per_processor

n	missing	distinct	Info	Mean	Gmd	.05	.10	.25	.50	.75	.90	.95
1008	12	12	0.977	8.572	4.694	2	4	6	8	10	14	16

Value	2	4	5	6	8	10	11	12	14	16	20	24
Frequency	81	118	8	182	198	189	1	93	69	42	3	24
Proportion	0.080	0.117	0.008	0.181	0.196	0.188	0.001	0.092	0.068	0.042	0.003	0.024

For the frequency table, variable is rounded to the nearest 0

Total_processor

n	missing	distinct	Info	Mean	Gmd
573	447	8	0.902	3.927	2.103

Value	1	2	4	5	6	8	10	12
Frequency	7	221	180	1	121	41	1	1
Proportion	0.012	0.386	0.314	0.002	0.211	0.072	0.002	0.002

For the frequency table, variable is rounded to the nearest 0

Execution_units

n	missing	distinct	Info	Mean	Gmd
573	447	5	0.734	6.998	2.478

Value	4	6	8	12	16
Frequency	193	3	350	3	24
Proportion	0.337	0.005	0.611	0.005	0.042

For the frequency table, variable is rounded to the nearest 0

Low_Power_Cores

n	missing	distinct	Info	Mean	Gmd
1020	0	2	0.124	0.08627	0.1653

Value	0	2
Frequency	976	44
Proportion	0.957	0.043

Energy_Efficient_Units

n	missing	distinct	Info	Sum	Mean	Gmd
1020	0	2	0.124	44	0.04314	0.08263

Threads

n	missing	distinct	Info	Mean	Gmd	.05	.10	.25	.50	.75	.90	.95
972	48	14	0.937	12.82	5.988	4	8	8	12	16	20	22

Value	2	4	5	6	8	12	14	16	18	20	22	24	28	32
Frequency	36	50	1	7	178	346	6	218	11	51	27	15	3	23
Proportion	0.037	0.051	0.001	0.007	0.183	0.356	0.006	0.224	0.011	0.052	0.028	0.015	0.003	0.024

For the frequency table, variable is rounded to the nearest 0

RAM_GB

n	missing	distinct	Info	Mean	Gmd	.05	.10	.25	.50	.75	.90	.95
1020	0	10	0.798	13.99	6.415	8	8	8	16	16	16	32

Value	2	4	8	12	16	18	32	36	48	64
Frequency	1	25	377	2	543	3	62	1	1	5
Proportion	0.001	0.025	0.370	0.002	0.532	0.003	0.061	0.001	0.001	0.005

For the frequency table, variable is rounded to the nearest 0

RAM_type

n	missing	distinct
998	22	12

Value	DDR3	DDR4	DDR5	DDR6	LPDDR3	LPDDR4	LPDDR4X	LPDDR5	LPDDR5X	LPDDRX4
Frequency	2	522	188	1	1	16	53	173	38	2
Proportion	0.002	0.523	0.188	0.001	0.001	0.016	0.053	0.173	0.038	0.002

Value	PDDR5X	Unified
Frequency	1	1
Proportion	0.001	0.001

Storage_capacity_GB

n	missing	distinct	Info	Mean	Gmd	.05	.10	.25	.50	.75	.90	.95
1020	0	10	0.664	627.7	265	256	512	512	512	512	1000	1000

Value	32	64	128	256	512	1000	1256	1512	2000	4000
Frequency	1	9	19	46	702	218	4	2	18	1
Proportion	0.001	0.009	0.019	0.045	0.688	0.214	0.004	0.002	0.018	0.001

For the frequency table, variable is rounded to the nearest 0

Storage_type

n	missing	distinct
1020	0	4

Value	Hard Disk	NVMe SSD	SSD	Hard Disk & SSD
Frequency	14	1	999	6
Proportion	0.014	0.001	0.979	0.006

Graphics_name

n	missing	distinct
1018	2	136

lowest :	10-Core GPU	10 Core GPU	14 Core GPU	18 Core GPU	30 Core GPU
highest:	Nvidia RTX 3050	NVIDIA RTX 4050	NVIDIA RTX A500	NVIDIA RTX NVIDIA T550	Radeon Pro 5500M

Graphics_brand

n	missing	distinct
1018	2	7

Value	Adreno	AMD	Apple	ARM	Intel	NVIDIA	Radeon
Frequency	1	151	18	7	486	354	1
Proportion	0.001	0.148	0.018	0.007	0.477	0.348	0.001

Graphics_GB

n	missing	distinct	Info	Mean	Gmd
368	652	6	0.871	5.94	2.556

Value	2	4	6	8	12	16
Frequency	4	174	87	80	12	11
Proportion	0.011	0.473	0.236	0.217	0.033	0.030

```
For the frequency table, variable is rounded to the nearest 0
```

Graphics_integreted

n	missing	distinct
1018	2	2

Value	False	True
Frequency	772	246
Proportion	0.758	0.242

Display_size_inches

n	missing	distinct	Info	Mean	Gmd	.05	.10	.25	.50	.75	.90	.95
1020	0	22	0.869	15.16	1.008	13.5	14.0	14.0	15.6	15.6	16.0	16.1

```
lowest : 11.6 12   12.4 13   13.3, highest: 16.1 16.2 17   17.3 18
```

Horizontal_pixel

n	missing	distinct	Info	Mean	Gmd	.05	.10	.25	.50	.75	.90	.95
1020	0	22	0.569	2036	340.2	1366	1920	1920	1920	1920	2560	2880

```
lowest : 1080 1200 1280 1366 1440, highest: 3024 3072 3200 3456 3840
```

Vertical_pixel

n	missing	distinct	Info	Mean	Gmd	.05	.10	.25	.50	.75	.90	.95
1020	0	22	0.767	1214	272.7	768	1080	1080	1080	1200	1800	1864

```
lowest :  768 1024 1080 1200 1280, highest: 2000 2160 2234 2400 2560
```

---

ppi

n	missing	distinct	Info	Mean	Gmd	.05	.10	.25	.50	.75	.90	.95
1020	0	62	0.902	157.2	30.93	127.3	141.2	141.2	141.2	161.7	212.3	242.6

```
lowest : 100.45 111.14 111.94 127.34 129.58, highest: 266.26 266.37 267.08 283.02 337.93
```

Touch_screen

n	missing	distinct
1020	0	2

Value	False	True
Frequency	904	116
Proportion	0.886	0.114

Operating_system

n	missing	distinct
1020	0	13

```
lowest : Android 11 OS Chrome OS     DOS 3.0 OS    DOS OS        jio
highest: Mac OS        Prime OS      Ubuntu OS     Windows 10 OS Windows 11 OS
```

Missing Values Proportion

```r
Summary.Table <- function(data){
  output <- data.frame(feature = names(data),
                       type = sapply(data, class),
           unique_value = sapply(data, function(x){if(!is.numeric(x)){length(unique(x))}
           else{"---"}}),
           num_missing = sapply(data, function(x){sum(is.na(x))}),
                       missing_rate = sapply(data, function(x) round(mean(is.na(x))*100, 2)))
  colnames(output) <- (c("feature", "type", "# of unique value",
                     "# of missing value", "missing rate(%)"))
  rownames(output) <- NULL
  return(output)
}
```

```r
library(DataExplorer)
library(gt)
Summary.Table(laptop) %>% gt()
```

feature	type	# of unique value	# of missing value	missing rate(%)
Brand	character	31	0	0.00
Price	integer	—	0	0.00
Rating	numeric	—	0	0.00
Processor_brand	character	7	0	0.00
Processor_name	character	36	0	0.00
Processor_variant	character	126	24	2.35
Processor_gen	numeric	—	129	12.65
Core_per_processor	numeric	—	12	1.18
Total_processor	numeric	—	447	43.82
Execution_units	numeric	—	447	43.82
Low_Power_Cores	numeric	—	0	0.00
Energy_Efficient_Units	integer	—	0	0.00
Threads	numeric	—	48	4.71
RAM_GB	integer	—	0	0.00
RAM_type	character	13	22	2.16
Storage_capacity_GB	integer	—	0	0.00
Storage_type	character	4	0	0.00
Graphics_name	character	137	2	0.20
Graphics_brand	character	8	2	0.20
Graphics_GB	numeric	—	652	63.92
Graphics_integreted	character	3	2	0.20
Display_size_inches	numeric	—	0	0.00
Horizontal_pixel	integer	—	0	0.00

Vertical_pixel	integer	—	0	0.00
ppi	numeric	—	0	0.00
Touch_screen	character	2	0	0.00
Operating_system	character	13	0	0.00

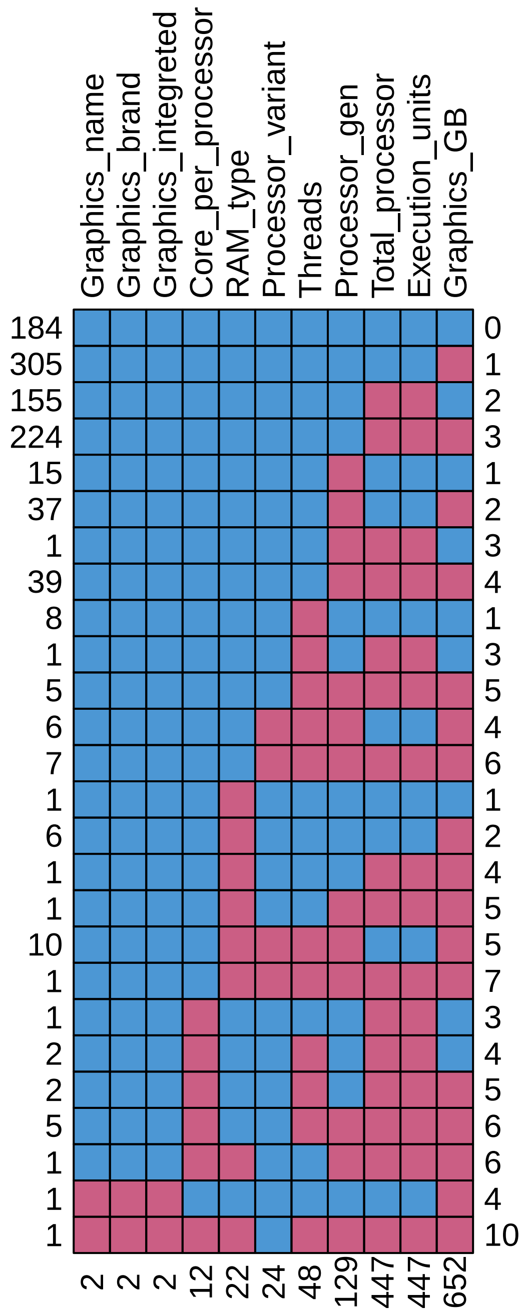
Missing Pattern

Next, we want to realize the missing pattern of laptop data.

```
# Customize md.pattern function originally in mice package.
library(mice)
md.pattern.mine <- function(x, plot = TRUE, rotate.names = FALSE, cex = 2.0){
  R <- is.na(x)
  # Calculate number of missing values in each variable.
  nmis <- colSums(R)
  # create a matrix, and column is ordered by nmis.
  R <- matrix(R[, order(nmis)], dim(x))
  # Transform rows to strings consisting of 0(non-missing) and 1(missing).
  pat <- apply(R, 1, function(x) paste(as.numeric(x), collapse = ""))
  # Sort rows by their missing patterns.
  sortR <- matrix(R[order(pat), ], dim(x))
  # Keep non-duplicated rows.
  mpat <- sortR[!duplicated(sortR), ]
  # obtain number of each missing pattern at LHS.
  rownames(mpat) <- table(pat)
  # abs(mpat - 1) obtain entries of matrix, (0, 1) means (missing, non-missing)
  # rowSums(mpat) obtain number of missing values in each row at RHS.
  r <- cbind(abs(mpat - 1), rowSums(mpat))
  # obtain number of missing values in each column at bottom center.
  # sum(nmis) sums values at bottom center.
  r <- rbind(r, c(nmis[order(nmis)], sum(nmis)))
  if (plot){
    op <- par(mar = rep(0, 4))
    on.exit(par(op))
    plot.new()
    if (is.null(dim(R))){
      R <- t(as.matrix(R))
    }
    R <- r[1:nrow(r) - 1, 1:ncol(r) - 1]
    if(rotate.names){
      adj <- c(0, 0.5)
      srt <- 90
      length_of_longest_colname <- max(nchar(colnames(r)))/2.6
      plot.window(xlim = c(-1, ncol(R) + 1),
                  ylim = c(-1, nrow(R) +
                         length_of_longest_colname), asp = 1)
    }
    else{
      adj <- c(0.5, 0)
      srt <- 0
      plot.window(xlim = c(-1, ncol(R) + 1),
                  ylim = c(-1, nrow(R) + 1), asp = 1)
    }
```

```r
    M <- cbind(c(row(R)), c(col(R))) - 1
    shade <- ifelse(R[nrow(R):1, ], mdc(1), mdc(2))
    rect(M[, 2], M[, 1], M[, 2] + 1, M[, 1] + 1, col = shade)
    for(i in 1:ncol(R)){
      # Show colnames at the top center.
      text(i - 0.5, nrow(R) + 0.3, colnames(r)[i], adj = adj,
          srt = srt, cex = cex)
      # Show number of missing values for each variable at the bottom center.
      text(i - 0.5, -0.8, nmis[order(nmis)][i], cex = cex, srt = srt)
    }
    for(i in 1:nrow(R)){
      # Show number of missing values for each subjects at the RHS.
      text(ncol(R) + 0.3, i - 0.5, r[(nrow(r) - 1):1, ncol(r)][i],
          adj = 0, cex = cex)
      # Show number of subjects for each kind of missing pattern at the LHS.
      text(-0.3, i - 0.5, rownames(r)[(nrow(r) - 1):1][i], adj = 1, cex = cex)
    }
    # show total number of missing entries.
    # text(ncol(R) + 1.3, -0.6, r[nrow(r), ncol(r)], cex = cex)
    # return(r)
  }
  else {
    return(r)
  }
}
```

```r
library(dplyr)
mis_col <- colnames(laptop)[profile_missing(laptop)$num_missing > 0]
laptop[,mis_col] %>%
  md.pattern.mine(rotate.names = T, cex = 1.0)
```



From the missing pattern plot, we can observe that
1. The missing values of “Total_processor” and “Execution_units” occur with each other.

Exploratory Data Analysis (EDA)

Numerical Columns

First, we look at numerical variables.

```
numerical_columns <- laptop %>%
  select(where(is.numeric)) %>%
  colnames()
numerical_columns
```

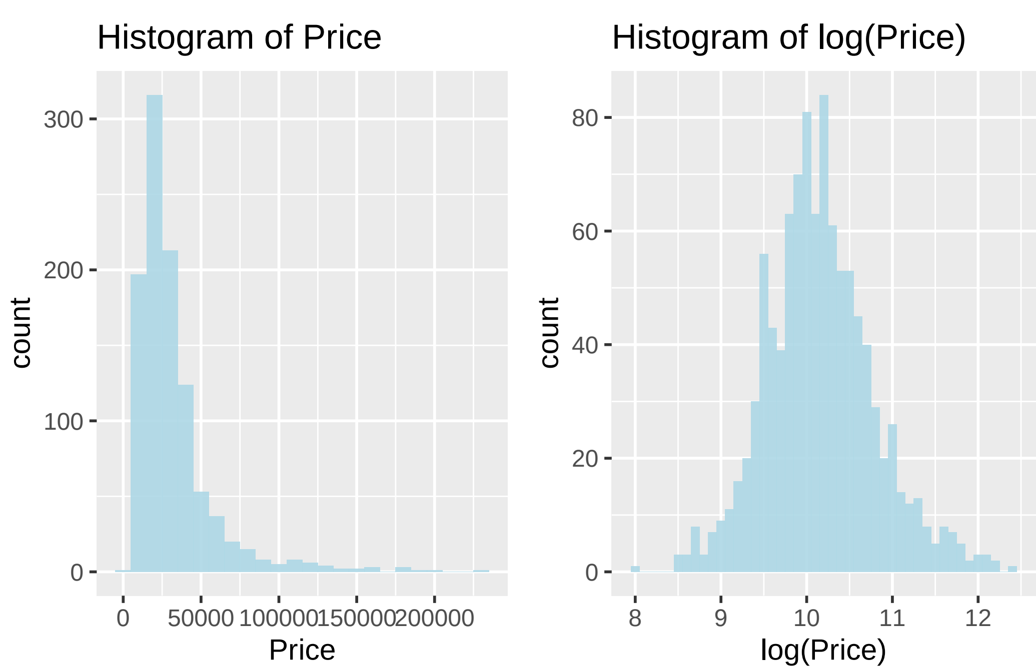
```
 [1] "Price"                  "Rating"                 "Processor_gen"
 [4] "Core_per_processor"     "Total_processor"        "Execution_units"
 [7] "Low_Power_Cores"        "Energy_Efficient_Units" "Threads"
[10] "RAM_GB"                 "Storage_capacity_GB"    "Graphics_GB"
[13] "Display_size_inches"    "Horizontal_pixel"       "Vertical_pixel"
[16] "ppi"
```

Next, we look at the outcome variable `Price`. Because the price in this data is INR (Indian Rupees), so we multiply 0.39, the exchange rate of TWD to INR.

```r
laptop$Price <- laptop$Price * 0.39
```

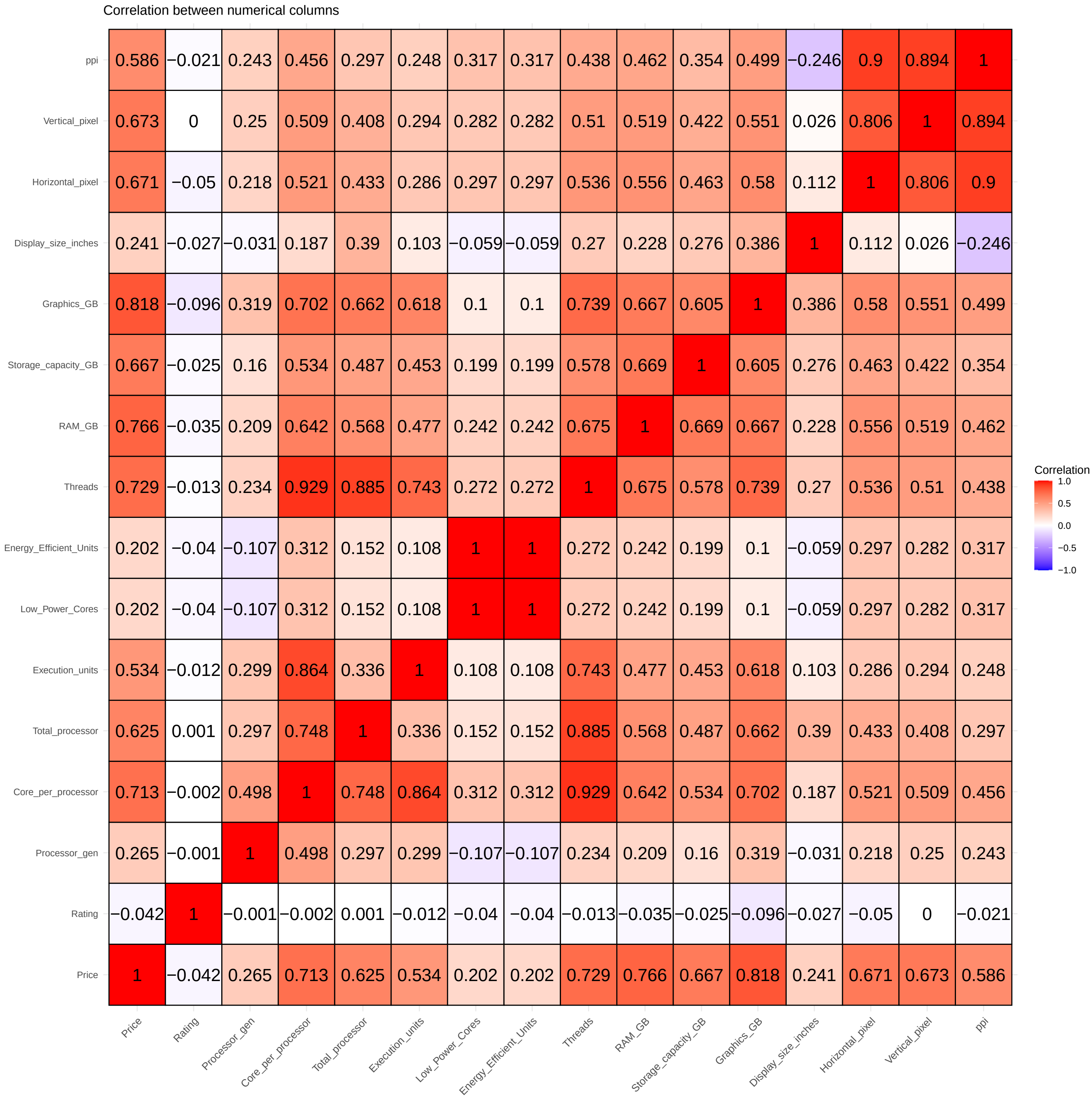
Because the original price is extremely skewed to the right, we take log() to make it more symmetric. Below are histograms of Price and log(Price).

```r
library(ggplot2)
library(gridExtra)
plot1 <- laptop %>%
  ggplot(aes(x = Price))+
  geom_histogram(binwidth = 10000, fill="lightblue",alpha=0.9)+
  ggtitle("Histogram of Price")

plot2 <- laptop %>%
  ggplot(aes(x = log(Price)))+
  geom_histogram(binwidth = 0.1, fill="lightblue",alpha=0.9)+
  ggtitle("Histogram of log(Price)")

grid.arrange(grobs = list(plot1, plot2), ncol = 2, nrow = 1)
```



Next, we look at the correlation between all numerical features.

```r
library(reshape2)
library(tidyverse)
# Compute the correlation matrix using pairwise deletion.
cor_matrix <- round(cor(laptop[,numerical_columns], use = "pairwise.complete.obs"), 3)

# Melt the correlation matrix for ggplot2
melted_cor_matrix <- melt(cor_matrix)

# Plotting with ggplot2
ggplot(data = melted_cor_matrix, aes(x = Var1, y = Var2, fill = value)) +
  geom_tile(color = "black", linewidth = 0.5) +
  scale_fill_gradient2(low = "blue", high = "red", mid = "white",
                       midpoint = 0, limit = c(-1, 1), space = "Lab",
                       name="Correlation") +
  geom_text(aes(label = value), color = "black", size = 6) +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, vjust = 1,
                                   size = 10, hjust = 1)) +
  labs(title = "Correlation between numerical columns", x = "", y = "")
```



From the correlation heatmap, we can discover that
1. "ppi", "Vertical_pixel" and "Horizontal_pixel" are highly correlated. We will keep "ppi", and delete "Vertical_pixel" and "Horizontal_pixel".
2. "Energy_Efficient_Units" and "Low_Power_Cores" are perfectly correlated ($\rho = 1$).
We will keep "Energy_Efficient_Units" and delete "Low_Power_Cores". 3. Rating has low correlation with Price, so we will also delete it.

Also, we think that Processor_gen has wrong information about its meaning, we will also delete it.

```
laptop <- subset(laptop, select = -c(Rating, Processor_gen, Low_Power_Cores,
                                      Energy_Efficient_Units, Vertical_pixel,
                                      Horizontal_pixel))
```

Categorical Columns

We look at each categorical variables.

```
# Obtain column names of categorical variables.
cat_cols <- laptop %>%
  select(where(is.character)) %>%
  colnames()
cat_cols
```

```
 [1] "Brand"             "Processor_brand"    "Processor_name"
 [4] "Processor_variant" "RAM_type"           "Storage_type"
 [7] "Graphics_name"     "Graphics_brand"     "Graphics_integreted"
[10] "Touch_screen"      "Operating_system"
```

1. Brand

```
sort(table(laptop$Brand))
```

```
     ASUS  Colorful     iBall       Jio   Ninkear     Razer     Tecno    Walker
        1         1         1         1         1         1         1         1
      AXL    Huawei   Fujitsu Primebook     Wings   Ultimus     Avita  Gigabyte
        2         2         3         3         3         5         6         6
    Honor    Xiaomi        LG Microsoft Zebronics     Chuwi     Apple   Infinix
        6         6         7         7         7         8        20        20
  Samsung      Acer       MSI      Dell      Asus        HP    Lenovo
       32        69        97       116       157       213       217
```

Because there is typing error about "ASUS" and "Asus", we unify them to "ASUS".

```
laptop[laptop == "Asus"] <- "ASUS"
```

2. Processor_brand

```
sort(table(laptop$Processor_brand))
```

```
HiSilicon Microsoft  Qualcomm  MediaTek     Apple       AMD     Intel
        1         1         1         7        18       250       742
```

3. Processor_name

```
sort(table(laptop$Processor_name))
```

```
              AMD Athlon Pro            AMD Athlon Silver
                           1                            1
                    Apple M1          eration Intel Core
                           1                            1
HiSilicon  Kirin 9006C 9006C             Intel Atom Quad
```

```
                       1                            1
      Intel Celeron Dual                  Intel Core 3
                       1                            1
    Intel Core I3-1115G4                Intel Pentium 
                       1                            1
      Intel Pentium Gold                     Mediatek
                       1                            1
       Microsoft SQ1 SQ1              Qualcomm X Elite
                       1                            1
    Apple M2 Apple M2 Chip                Apple M3 Max
                       2                            2
                 Intel            Intel Pentium Silver
                       2                            2
                Apple M2                  Apple M3 Pro
                       3                            3
            Intel Core 7                     MediaTek
                       3                            3
      MediaTek Kompanio                    AMD Athlon
                       3                            4
            Intel Core 5                   AMD Ryzen 9
                       4                            7
                Apple M3                   AMD Ryzen 3
                       7                           31
          Intel Celeron               Intel Core Ultra
                      36                           44
           Intel Core i9                   AMD Ryzen 7
                      49                           87
           Intel Core i3                   AMD Ryzen 5
                     114                          119
           Intel Core i7                 Intel Core i5
                     159                          322
```

Because there are so many names that have only 1 subject, we have to modify `Processor_name` by hand.

```
laptop[laptop == "AMD Athlon Pro"] <- "AMD Athlon"
laptop[laptop == "AMD Athlon Silver"] <- "AMD Athlon"
laptop[laptop == "eration Intel Core"] <- "Intel Core i5"
laptop[laptop == "Intel Celeron Dual"] <- "Intel Celeron"
laptop[laptop == "Intel Core 3"] <- "Intel Core i3"
laptop[laptop == "Intel Core I3-1115G4"] <- "Intel Core i3"
laptop[laptop == "Intel Pentium "] <- "Intel Pentium"
laptop[laptop == "Intel Pentium Gold"] <- "Intel Pentium"
laptop[laptop == "Mediatek"] <- "MediaTek"
laptop[laptop == "Apple M2 Apple M2 Chip"] <- "Apple M2"
laptop[laptop == "Apple M3 Max"] <- "Apple M3"
laptop[laptop == "Intel  "] <- "Intel Celeron"
laptop[laptop == "Intel Pentium Silver"] <- "Intel Pentium"
laptop[laptop == "Apple M3 Pro"] <- "Apple M3"
laptop[laptop == "Intel Core 7"] <- "Intel Core i7"
laptop[laptop == "MediaTek Kompanio"] <- "MediaTek"
laptop[laptop == "Intel Core 5"] <- "Intel Core i5"
laptop[laptop == "Intel Celeron "] <- "Intel Celeron"
```

4. Processor_variant

```r
length(unique(laptop$Processor_variant))
```

```
[1] 126
```

Since `Processor_variant` has 126 categories, we don't need to explore this feature. We will delete this column in the data preprocessing stage.

5. RAM_type

```r
sort(table(laptop$RAM_type))
```

```
   DDR6   LPDDR3   PDDR5X Unified     DDR3 LPDDRX4   LPDDR4 LPDDR5X LPDDR4X   LPDDR5
      1        1        1        1        2        2       16       38       53      173
   DDR5     DDR4
    188      522
```

Because there are some typing errors, we modify them by hand.

```r
laptop[laptop == "LPDDRX4"] <- "LPDDR4X"
laptop[laptop == "PDDR5X"] <- "LPDDR5X"
```

Also, there are 22 missing values in RAM_type, we surfed the internet, find the true RAM_type, and modified them by hand.

```r
which(is.na(laptop$RAM_type))
```

```
 [1]  18  84 117 164 176 203 204 295 296 348 349 350 351 403 552 649 652 657 664
[20] 728 774 903
```

```r
laptop[c(18,84,117,164,203,204,295,348,349,350,351) , "RAM_type"] <- "Unified"
laptop[176, "RAM_type"] <- "LPDDR5"
laptop[296, "RAM_type"] <- "DDR4"
laptop[403, "RAM_type"] <- "DDR4"
laptop[552, "RAM_type"] <- "DDR4"
laptop[c(649,652), "RAM_type"] <- "Unknown"
laptop[657, "RAM_type"] <- "DDR4"
laptop[664, "RAM_type"] <- "DDR4"
laptop[728, "RAM_type"] <- "DDR4"
laptop[774, "RAM_type"] <- "DDR4"
laptop[903, "RAM_type"] <- "LPDDR5"
```

6. Storage_type

```r
sort(table(laptop$Storage_type))
```

```
       NVMe SSD Hard Disk & SSD       Hard Disk             SSD
              1               6              14             999
```

7. Graphics_name

```r
length(unique(laptop$Graphics_name))
```

```
[1] 137
```

Since `Graphics_name` has 137 categories, we don't need to explore this feature. We will delete this column in the data preprocessing stage.

8. Graphics_brand

```r
sort(table(laptop$Graphics_brand))
```

```

Adreno Radeon    ARM  Apple    AMD NVIDIA  Intel 
     1      1      7     18    151    354    486 
```

Because there are 2 missing values in `Graphics_brand`, we modified them by hand.

```r
which(is.na(laptop$Graphics_brand))
```

```
[1] 649 999
```

```r
laptop[649, "Graphics_brand"] = "Unknown"
laptop[999, "Graphics_brand"] = "NVIDIA"
```

9. Graphics_integreted

```r
sort(table(laptop$Graphics_integreted))
```

```

 True False 
  246   772 
```

Because there are 2 missing values in `Graphics_integreted`, but we don't know the true value, we modified them by the majority class "False", then convert "True" to 1, "False" to 0.

```r
which(is.na(laptop$Graphics_integreted))
```

```
[1] 649 999
```

```r
laptop[c(649,999), "Graphics_integreted"] = "False"
laptop$Graphics_integreted <- ifelse(laptop$Graphics_integreted == "True", 1, 0)
```

10. Touch_screen

```r
sort(table(laptop$Graphics_integreted))
```

```

  1   0 
246 774 
```

We convert "True" to 1, "False" to 0.

```
laptop$Touch_screen <- ifelse(laptop$Touch_screen == "True", 1, 0)
```

11. Operating_system

```
sort(table(laptop$Operating_system))
```

```
   Android 11 OS       DOS 3.0 OS              jio         Linux OS 
               1                1                1                1 
Mac 10.15.3\t OS   Mac Catalina OS         Prime OS        Ubuntu OS 
               1                1                2                3 
       Chrome OS           DOS OS    Windows 10 OS           Mac OS 
              15               16               17               18 
   Windows 11 OS 
             943 
```

Inside the Operating_system, there are some subjects that are too detailed, so we modified them to general ones.

```
laptop[laptop == "Android 11 OS"] <- "Prime OS"
laptop[laptop == "DOS 3.0 OS"] <- "DOS OS"
laptop[laptop == "Mac 10.15.3\t OS"] <- "Mac OS"
laptop[laptop == "Mac Catalina OS"] <- "Mac OS"
```

After we modify some typing errors, we delete `Processor_variant`, `Graphics_name` for their high cardinalities, and delete `Processor_brand` since it's overlapped to another feature `Processor_name`.

```
laptop <- subset(laptop, select = -c(Processor_brand, Processor_variant, Graphics_name))
```

Data Preprocessing

Imputation

We use MICE imputation with method = 'pmm' to impute our missing data.

```
temp_data <- mice(laptop, m=5, maxit=100, meth='pmm', seed=500, printFlag = FALSE)
data <- complete(temp_data, 1)
```

Drop rare subjects

```
library(tidyverse)
value_counts <- data %>% group_by(Brand) %>%  summarise(count = n())%>% arrange(count)
data <- data %>% group_by(Brand) %>% filter(n() >= 6) %>% ungroup()
value_counts <- data %>% group_by(Processor_name) %>%  summarise(count = n())%>% arrange(count)
data <- data %>% group_by(Processor_name) %>% filter(n() >= 2) %>% ungroup()
value_counts <- data %>% group_by(Graphics_brand) %>%  summarise(count = n())%>% arrange(count)
data <- data %>% group_by(Graphics_brand) %>% filter(n() >= 4) %>% ungroup()
value_counts <- data %>% group_by(RAM_type) %>%  summarise(count = n())%>% arrange(count)
data <- data %>% group_by(RAM_type) %>% filter(n() >= 2) %>% ungroup()
```

```
value_counts <- data %>% group_by(Storage_type) %>%  summarise(count = n())%>% arrange(count)
data <- data %>% group_by(Storage_type) %>% filter(n() >= 2) %>% ungroup()
value_counts <- data %>% group_by(Operating_system) %>%  summarise(count = n())%>% arrange(count)
data <- data %>% group_by(Operating_system) %>% filter(n() >= 4) %>% ungroup()
```

Finally, we look at the data we will analyze later.

```
Summary.Table(data) %>% gt()
```

feature	type	# of unique value	# of missing value	missing rate(%)
Brand	character	17	0	0
Price	numeric	—	0	0
Processor_name	character	14	0	0
Core_per_processor	numeric	—	0	0
Total_processor	numeric	—	0	0
Execution_units	numeric	—	0	0
Threads	numeric	—	0	0
RAM_GB	integer	—	0	0
RAM_type	character	7	0	0
Storage_capacity_GB	integer	—	0	0
Storage_type	character	3	0	0
Graphics_brand	character	4	0	0
Graphics_GB	numeric	—	0	0
Graphics_integreted	numeric	—	0	0
Display_size_inches	numeric	—	0	0
ppi	numeric	—	0	0
Touch_screen	numeric	—	0	0
Operating_system	character	5	0	0

Split into training and testing dataset

We split the dataset into training and testing data.

```
sample_indices <- sample(seq_len(nrow(data)), size = 0.8 * nrow(data))
trainData <- data[sample_indices, ]
testData <- data[-sample_indices, ]
cat("Training set dimensions:", dim(trainData), "\n")
```

```
Training set dimensions: 784 18
```

```
cat("Testing set dimensions:", dim(testData), "\n")
```

```
Testing set dimensions: 197 18
```

Model Fitting

```r
MAE <- function(pred, true) return(mean(abs(pred-true)))
RMSE <- function(pred, true) return(sqrt(mean((pred-true)^2)))
```

Linear Regression

```r
fit <- lm(log(Price) ~ ., data = trainData)
summary(fit)
```

```
Call:
lm(formula = log(Price) ~ ., data = trainData)

Residuals:
     Min       1Q   Median       3Q      Max
-0.61850 -0.09007 -0.01030  0.07890  0.73759

Coefficients: (3 not defined because of singularities)
                                Estimate Std. Error t value Pr(>|t|)
(Intercept)                    8.043e+00  1.972e-01  40.780  < 2e-16 ***
BrandApple                     7.007e-01  1.824e-01   3.841 0.000133 ***
BrandASUS                      1.183e-01  2.576e-02   4.591 5.20e-06 ***
BrandAvita                    -3.536e-01  7.660e-02  -4.616 4.62e-06 ***
BrandChuwi                    -2.532e-01  7.956e-02  -3.182 0.001523 **
BrandDell                      1.678e-01  2.749e-02   6.105 1.67e-09 ***
BrandGigabyte                  1.152e-01  7.431e-02   1.550 0.121579
BrandHonor                     1.267e-02  8.468e-02   0.150 0.881066
BrandHP                        1.774e-01  2.458e-02   7.216 1.34e-12 ***
BrandInfinix                  -2.763e-01  5.556e-02  -4.972 8.25e-07 ***
BrandLenovo                    1.231e-01  2.468e-02   4.988 7.63e-07 ***
BrandLG                        4.690e-01  6.529e-02   7.184 1.67e-12 ***
BrandMicrosoft                 3.498e-01  7.366e-02   4.749 2.46e-06 ***
BrandMSI                       8.408e-02  2.883e-02   2.916 0.003650 **
BrandSamsung                   2.380e-01  4.528e-02   5.257 1.92e-07 ***
BrandXiaomi                   -4.030e-01  8.609e-02  -4.682 3.39e-06 ***
BrandZebronics                -3.397e-01  7.005e-02  -4.849 1.52e-06 ***
Processor_nameAMD Ryzen 3      1.578e-01  1.222e-01   1.291 0.197083
Processor_nameAMD Ryzen 5      4.142e-01  1.208e-01   3.428 0.000642 ***
Processor_nameAMD Ryzen 7      5.822e-01  1.255e-01   4.638 4.17e-06 ***
Processor_nameAMD Ryzen 9      7.094e-01  1.550e-01   4.578 5.51e-06 ***
Processor_nameApple M2         5.866e-01  2.223e-01   2.639 0.008493 **
Processor_nameApple M3         7.041e-01  2.120e-01   3.321 0.000942 ***
Processor_nameIntel Celeron   -1.185e-01  1.214e-01  -0.977 0.329089
Processor_nameIntel Core i3    1.405e-01  1.226e-01   1.146 0.252139
Processor_nameIntel Core i5    3.932e-01  1.229e-01   3.199 0.001440 **
Processor_nameIntel Core i7    5.747e-01  1.247e-01   4.610 4.76e-06 ***
Processor_nameIntel Core i9    6.302e-01  1.322e-01   4.765 2.27e-06 ***
Processor_nameIntel Core Ultra 4.666e-01  1.313e-01   3.553 0.000406 ***
Processor_nameIntel Pentium    2.058e-02  1.414e-01   0.145 0.884375
Core_per_processor             3.008e-02  3.829e-02   0.786 0.432317
Total_processor               -3.407e-02  3.765e-02  -0.905 0.365761
Execution_units               -2.391e-02  7.788e-03  -3.070 0.002219 **
```

```
Threads                        7.267e-03  3.732e-02   0.195 0.845651
RAM_GB                         1.167e-02  1.278e-03   9.129  < 2e-16 ***
RAM_typeDDR5                   1.483e-01  2.107e-02   7.039 4.48e-12 ***
RAM_typeLPDDR4                -1.056e-01  7.338e-02  -1.439 0.150519
RAM_typeLPDDR4X                8.405e-02  3.878e-02   2.168 0.030515 *
RAM_typeLPDDR5                 8.390e-02  1.866e-02   4.497 8.02e-06 ***
RAM_typeLPDDR5X                1.866e-01  4.064e-02   4.592 5.16e-06 ***
RAM_typeUnified                       NA         NA      NA       NA
Storage_capacity_GB            1.223e-04  2.799e-05   4.368 1.43e-05 ***
Storage_type SSD               4.093e-02  1.099e-01   0.372 0.709715
Storage_typeHard Disk & SSD    3.664e-02  1.306e-01   0.281 0.779141
Graphics_brandApple                   NA         NA      NA       NA
Graphics_brandIntel            1.191e-01  3.216e-02   3.705 0.000227 ***
Graphics_brandNVIDIA           2.606e-01  2.708e-02   9.625  < 2e-16 ***
Graphics_GB                    6.058e-02  4.787e-03  12.656  < 2e-16 ***
Graphics_integreted            4.837e-03  1.831e-02   0.264 0.791690
Display_size_inches            1.855e-02  8.431e-03   2.201 0.028071 *
ppi                            2.257e-03  2.721e-04   8.298 5.11e-16 ***
Touch_screen                   2.658e-01  2.240e-02  11.870  < 2e-16 ***
Operating_systemDOS OS        -1.733e-02  9.977e-02  -0.174 0.862132
Operating_systemMac OS                NA         NA      NA       NA
Operating_systemWindows 10 OS  4.197e-01  1.034e-01   4.061 5.42e-05 ***
Operating_systemWindows 11 OS  1.071e-01  8.866e-02   1.208 0.227270
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.1561 on 731 degrees of freedom
Multiple R-squared:  0.9383,    Adjusted R-squared:  0.9339
F-statistic: 213.8 on 52 and 731 DF,  p-value: < 2.2e-16
```

```r
lr_pred <- exp(predict(fit, newdata = testData))
```

```
Warning in predict.lm(fit, newdata = testData): prediction from rank-deficient
fit; attr(*, "non-estim") has doubtful cases
```

```r
lr_mae <- MAE(lr_pred, testData$Price)
lr_rmse <- RMSE(lr_pred, testData$Price)
cat("LR MAE:", lr_mae, "RMSE", lr_rmse)
```

```
LR MAE: 4418.98 RMSE 9370.868
```

Support Vector Regression

```r
library(e1071)
svr_model <- svm(log(Price) ~ ., data = trainData, kernel = "linear")
svr_pred <- exp(predict(svr_model, newdata = testData))
svr_mae <- MAE(svr_pred, testData$Price)
svr_rmse <- RMSE(svr_pred, testData$Price)
cat("SVR MAE:", svr_mae, "RMSE", svr_rmse)
```

```
SVR MAE: 4339.883 RMSE 9195.034
```

Random Forest Regressor

```
library(randomForest)
rf_model <- randomForest(x = trainData[,-2], y = trainData$Price, importance = TRUE)
rf_pred <- predict(rf_model, newdata = testData[,-2])
rf_mae <- MAE(rf_pred, testData$Price)
rf_rmse <- RMSE(rf_pred, testData$Price)
cat("RF MAE:", rf_mae, "RMSE", rf_rmse)
```

```
RF MAE: 4543.193 RMSE 9560.965
```

XGBoost

XGBoost and LightGBM need to onehot encoding for categorical data !

```
data_numerical <- data %>% select(where(is.numeric))
data_categorical <- data %>% select(where(is.character))
onehot.matrix <- model.matrix(~ 0 + Brand + Processor_name + RAM_type + Graphics_brand
                              + Operating_system, data = data_categorical)
data_ML <- cbind(data_numerical, onehot.matrix)
trainData <- data_ML[sample_indices, ]
testData <- data_ML[-sample_indices, ]
```

```
library(xgboost)
features_train <- trainData[, !colnames(trainData) %in% "Price"]
features_test <- testData[, !colnames(testData) %in% "Price"]
target_train <- trainData$Price
target_test <- testData$Price

dtrain <- xgb.DMatrix(data = as.matrix(features_train), label = log(target_train))
dtest <- xgb.DMatrix(data = as.matrix(features_test), label = log(target_test))
xgb_model <- xgboost(data = dtrain, nrounds = 100, objective = "reg:squarederror", verbose = 0)
xgb_pred <- exp(predict(xgb_model, dtest))
xgb_mae <- MAE(xgb_pred, testData$Price)
xgb_rmse <- RMSE(xgb_pred, testData$Price)
cat("XGB MAE:", xgb_mae, "RMSE", xgb_rmse)
```

```
XGB MAE: 4693.1 RMSE 10426.95
```

LightGBM

```
library(lightgbm)
features_train <- trainData[, !colnames(trainData) %in% "Price"]
features_test <- testData[, !colnames(testData) %in% "Price"]
target_train <- trainData$Price
target_test <- testData$Price

dtrain <- lgb.Dataset(data = as.matrix(features_train), label = log(target_train))
dtest <- lgb.Dataset(data = as.matrix(features_test), label = log(target_test))
```

```r
params <- list(objective = "regression", metric = "rmse")
lightgbm_model <- lgb.train(params = params, data = dtrain,
                            nrounds = 100, verbose = -1)
lgbm_pred <- exp(predict(lightgbm_model, newdata = as.matrix(features_test)))
lgbm_mae <- MAE(lgbm_pred, testData$Price)
lgbm_rmse <- RMSE(lgbm_pred, testData$Price)
cat("LightGBM MAE:", lgbm_mae, "RMSE", lgbm_rmse)
```

```
LightGBM MAE: 4596.927 RMSE 9574.013
```

Catboost

```r
library(catboost)
features_train <- trainData[, !colnames(trainData) %in% "Price"]
features_test <- testData[, !colnames(testData) %in% "Price"]
target_train <- trainData$Price
target_test <- testData$Price

train_pool <- catboost.load_pool(data = features_train, label = target_train)
fit_params <- list(loss_function = 'RMSE', iterations = 100, logging_level = "Silent")
model_cat <- catboost.train(train_pool, params = fit_params)

test_pool <- catboost.load_pool(data = features_test, label = target_test)
cat_pred <- catboost.predict(model_cat, test_pool)
cat_mae <- MAE(cat_pred, testData$Price)
cat_rmse <- RMSE(cat_pred, testData$Price)
cat("Catboost MAE:", cat_mae, "RMSE", cat_rmse)
```

```
Catboost MAE: 4554.289 RMSE 9642.532
```

Conclusion

Let's see which model predicts the best.

```r
rmse <- c(lr_rmse, svr_rmse, rf_rmse, xgb_rmse, lgbm_rmse, cat_rmse)
mae <- c(lr_mae, svr_mae, rf_mae, xgb_mae, lgbm_mae, cat_mae)
performance <- round(rbind(rmse, mae),2)
colnames(performance) <- c("LR", "SVR", "RF", "XGB", "LightGBM", "Catboost")
data.frame(performance) %>% gt(rownames_to_stub = TRUE)
```

	LR	SVR	RF	XGB	LightGBM	Catboost
rmse	9370.87	9195.03	9560.96	10426.95	9574.01	9642.53
mae	4418.98	4339.88	4543.19	4693.10	4596.93	4554.29

Next, we look at Feature importance of Random Forest, XGBoost and LightGBM.

```r
# Random Forest
importance_scores <- importance(rf_model)
importance_df <- data.frame(Importance = importance_scores[, "%IncMSE"],
                            Feature = rownames(importance_scores))
importance_df <- importance_df[order(-importance_df$Importance), ]
top_10_features.rf <- importance_df[1:10,]$Feature

# XGBoost
importance_matrix <- xgb.importance(feature_names = colnames(features_train), model = xgb_model)

# Sorting the importance matrix by gain in descending order
top_features <- importance_matrix[order(-importance_matrix$Gain),]
# Selecting the top 10 features
top_10_features.xgb <- head(top_features, 10)$Feature

# LightGBM
feature_importance <- lgb.importance(model = lightgbm_model)
sorted_feature_importance <- feature_importance[order(-feature_importance$Gain), ]
top_10_features.lgb <- head(sorted_feature_importance, 10)$Feature

# Catboost
feature_importance_cat <- catboost.get_feature_importance(model_cat, train_pool)
sorted_feature_importance_cat <- rownames(feature_importance_cat)[order(-feature_importance_cat)]
top_10_features.cat <- head(sorted_feature_importance_cat, 10)

importance_comparison <- cbind(top_10_features.rf, top_10_features.xgb,
                               top_10_features.lgb, top_10_features.cat)
colnames(importance_comparison) <- c("RF", "XGB", "LightGBM", "Catboost")
rownames(importance_comparison) <- c(1:10)
data.frame(importance_comparison) %>% gt(rownames_to_stub = TRUE)
```

	RF	XGB	LightGBM	Catboost
1	ppi	ppi	Graphics_GB	Graphics_GB
2	Graphics_GB	Graphics_GB	Threads	ppi
3	Processor_name	Core_per_processor	RAM_GB	RAM_GB
4	Touch_screen	Threads	ppi	Core_per_processor
5	RAM_GB	RAM_GB	Touch_screen	Storage_capacity_GB
6	Core_per_processor	Touch_screen	Core_per_processor	Touch_screen
7	Threads	Storage_capacity_GB	Processor_nameIntel_Core_i7	Display_size_inches
8	RAM_type	Processor_nameIntel Core i7	Storage_capacity_GB	Processor_nameIntel Core i7
9	Graphics_brand	Graphics_brandNVIDIA	Processor_nameIntel_Core_i5	Processor_nameIntel Core i9
10	Graphics_integreted	Total_processor	Graphics_brandNVIDIA	Threads

From table table above, we can conclude that the top 3 features to predict the price of a laptop are:
1. Graphics_GB
2. ppi
3. RAM_GB